

Near-Policy: Accelerating On-Policy Distillation via Asynchronous Generation and Selective Packing

Miao Rang^{*1} Zhenni Bi^{*1} Hang Zhou² Kai Han¹ Xuechun Wang¹
 An Xiao¹ Xinghao Chen¹ Yunhe Wang¹ Hanling Chen^{1†}
¹Huawei Technologies, ²Tianjin University

Abstract

Standard knowledge distillation for autoregressive models often suffers from distribution mismatch. While on-policy methods mitigate this by leveraging student-generated outputs, they rely on computationally expensive Reinforcement Learning (RL) frameworks. To improve efficiency, we propose **Near-Policy Distillation (NPD)**, an asynchronous approach that decouples student generation from training. This reformulation enables Supervised Fine-Tuning (SFT) with sequence packing. However, asynchronous updates inevitably introduce policy lag and sample noise, which can cause the behavior to drift from near-policy toward off-policy. To counteract this without sacrificing efficiency, NPD integrates sparse student updates and the Δ -IFD filtering mechanism, a heuristic sample selection mechanism that empirically stabilizes the optimization trajectory. By filtering extreme out-of-distribution samples, Δ -IFD prevents noise from dominating the gradients, ensuring updates remain within a safe proximal learning zone. Empirically, the NPD framework achieves a $8.1\times$ speedup over on-policy baselines and outperforms SFT by **8.09%**. Crucially, by effectively narrowing the exploration space for subsequent RL, our method enables openPangu-Embedded-1B to reach a state-of-the-art score of **68.73%**, outperforming the substantially larger Qwen3-1.7B. Codes will be released soon.

1 Introduction

Large Language Models (LLMs) have revolutionized artificial intelligence by leveraging self-attention mechanisms and massive-scale pre-training to capture hierarchical linguistic patterns, semantic relationships, and cross-domain knowledge Achiam et al. [2023]. Open-sourced models such as LLaMA Touvron et al. [2023], DeepSeek Liu et al. [2024], Qwen Bai et al. [2023], Yang et al. [2025], and openPangu Chen et al. [2025], Tang et al. [2025] have demonstrated exceptional capabilities in complex tasks, including reasoning and multilingual understanding. However, their success relies heavily on colossal parameter counts (e.g., DeepSeek-V3 Liu et al. [2024] possesses 671B parameters) and extensive computational resources. The prohibitive energy consumption and hardware requirements of these models present critical barriers to real-world deployment, particularly in scenarios demanding on-device processing, privacy preservation, or low-latency responsiveness.

Standard Knowledge Distillation (KD) approaches Hinton et al. [2015], Sanh et al. [2019] typically utilize Kullback-Leibler Divergence (KLD) loss to align the student’s output distribution with the teacher’s on a fixed dataset. While this inherits the architectural efficiency of Supervised Fine-Tuning (SFT) and improves performance over purely supervised methods, it suffers from a fundamental *distribution mismatch*: the student is trained on teacher-forced sequences but generates autoregressive sequences during inference. To mitigate this, on-policy methods Agarwal et al. [2024a] train the

^{*}Equal contribution.

[†]Corresponding author. Email: chenhanling@huawei.com

student on its own Student-Generated Outputs (SGOs). Although effective in correcting distribution shift, these methods rely on synchronous RL-like frameworks that necessitate simultaneous teacher-student inference. Consequently, they inherit the inefficiencies of RL, specifically the inability to utilize sequence packing and the low token utilization rate caused by continuous generation.

To overcome these limitations, we propose Near-Policy Distillation. This asynchronous framework fundamentally decouples SGOs generation from the gradient update process. By pre-computing SGOs and teacher logits, NPD approximates the distribution-matching benefits of on-policy distillation while reformulating the training phase as SFT. Crucially, this allows us to leverage sequence packing, thereby maximizing training throughput. However, the asynchronous nature of NPD introduces a new challenge: *policy lag*. Since the data generation policy differs from the current training policy, samples may become noisy or drift toward off-policy distributions, potentially destabilizing the student’s alignment.

To counteract this drift without sacrificing efficiency, we employ a dual strategy. First, we implement sparse student updates, ensuring that the generation policy never deviates significantly from the training policy. Second, we introduce a dynamic filtering mechanism based on Δ -**IFD**. By leveraging the Instruction-Following Difficulty (IFD) Li et al. [2024] metric, we estimate the discrepancy between the teacher and the evolving student policy. This mechanism acts as a gatekeeper, empirically stabilizing the optimization trajectory by filtering out extreme out-of-distribution samples. This prevents high-variance noise from dominating the gradients and ensures the student updates within a safe proximal learning zone, maintaining a near-policy state despite the asynchronous updates. Empirically, the NPD framework yields an **8.09%** performance improvement across the distillation phase.

Finally, we investigate the synergy between distillation and reinforcement learning. We demonstrate that models trained via NPD serve as superior initialization points for RL. By effectively constraining the exploration space, NPD enables the subsequent RL stage to achieve significantly higher precision.

Our main contributions are summarized as follows:

- We propose **Near-Policy Distillation**, an asynchronous framework that decouples generation from training to enable sequence packing, achieving the performance benefits of on-policy distillation with the efficiency of off-policy training (**$8.1\times$ speedup**).
- We address the policy lag inherent in asynchronous updates by employing sparse student updates alongside the Δ -**IFD** filtering mechanism. By filtering extreme out-of-distribution samples, it prevents noise-dominated gradients and keeps updates within a safe proximal learning zone, thereby empirically stabilizing the optimization trajectory.
- We demonstrate that NPD unlocks the full potential of subsequent RL by narrowing the search space. This powerful combination enables our openPangu Embedded-1B model to achieve a state-of-the-art score of **68.73%**, surpassing the substantially larger Qwen3-1.7B (63.69%).

2 Related Works

Knowledge Distillation (KD) Bucila et al. [2006], Hinton et al. [2015] transfers capabilities from a high-capacity teacher to a compact student. In the context of autoregressive LLMs, methodologies primarily diverge into RL-based (sample-wise distillation) and SFT-based (sequence-packed distillation) paradigms.

In RL-style (sample-wise) distillation, the student is optimized under its *own* trajectory distribution to reduce the training–inference mismatch that arises when learning from teacher-generated sequences but deploying on student-generated ones. MiniLLM Gu et al. [2025] argues that forward-KL distillation (maximum likelihood under the teacher distribution) may over-cover low-probability regions and harm generation quality, and therefore adopts a reverse-KL objective with an effective on-policy training procedure Gu et al. [2025]. GKD Agarwal et al. [2024b] makes this principle explicit by sampling student trajectories and distilling teacher token-level guidance on them, ensuring updates target the inference-time state distribution Agarwal et al. [2024b]. Building on this line, DistiLLM and DistiLLM-2 Ko et al. [2024, 2025] improve objective design (e.g., skew-KL and contrastive formulations) and data efficiency via adaptive reuse of student-generated samples, reducing the cost

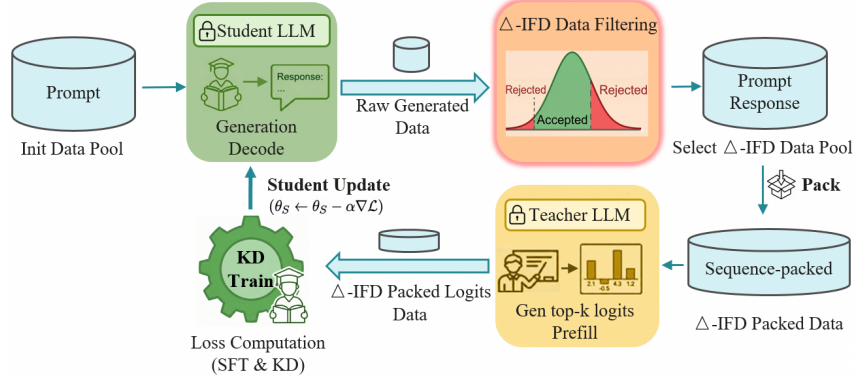


Figure 1: **Overview of the Near-Policy Distillation Loop with Δ -IFD data selection and student updates.** The pipeline consists of three phases: (1) **Generation & Filtering:** The Student LLM generates raw responses from the initial prompt pool, which are then filtered using the Δ -IFD metric. Samples falling outside the Δ_{IFD} threshold are rejected to ensure data quality. (2) **Teacher Annotation:** The selected samples are sequence-packed, and the Teacher LLM performs a prefill step to generate top- k logits. (3) **Student Updates:** The Student LLM is updated using a combined loss of SFT and KD derived from the teacher’s logits.

of online sampling and teacher inference Ko et al. [2024, 2025]. Overall, on-policy distillation better corrects mismatch under the student’s distribution, but its throughput is often limited by online generation and teacher evaluation overhead.

SFT-based (sequence-packed) distillation follows a classical offline learning setup. Methods such as standard KD Hinton et al. [2015] and SeqKD Kim and Rush [2016b] train the student to match teacher logits and/or learn from teacher-generated sequences on a fixed corpus. This purely supervised formulation is highly scalable and naturally compatible with throughput optimizations such as sequence packing. However, because training conditions on reference trajectories rather than student rollouts, it does not explicitly correct inference-time state-distribution shift and may still suffer from exposure bias and error accumulation. To bridge this gap, we propose Near-Policy Distillation. By decoupling generation from optimization via an asynchronous pipeline, NPD distills teacher guidance on student-generated trajectories while retaining SFT-style sequence packing for high throughput, improving efficiency and maintaining alignment with the evolving student policy.

3 Method

In this section, we detail the NPD framework (Fig 1). We first define the asynchronous paradigm, which decouples generation from training to maximize throughput. Subsequently, we introduce the Δ -IFD mechanism designed to curate high-value samples by leveraging teacher-student discrepancy. Finally, we discuss the optimization objective and the synergistic integration of NPD with Reinforcement Learning.

3.1 Near-Policy Knowledge Distillation

As outlined in Algorithm 1, Our core strategy involves *decoupling* the student’s generation from the gradient update loop. This design bridges the distribution mismatch—a key advantage of on-policy methods—while unlocking the high training throughput typical of SFT.

The workflow proceeds in three streamlined stages: (1) **Generation:** We utilize high-throughput batch inference to generate student responses; (2) **Gen Logits:** These sequences are packed and fed into the teacher model, which computes logits via a highly efficient parallel prefill process; (3) **Optimization:** Finally, distillation is performed within a standard SFT architecture utilizing an auxiliary KD loss.

Student-Driven Response Generation. To align the training distribution with the student’s evolving policy, we employ the current student model (parameterized by θ_S) to generate responses for the prompts dataset $\mathcal{D}_x$. We leverage high-performance inference engines (e.g., vLLM) to perform batch inference, producing a set of student-generated trajectories $\mathcal{D}_S = \{(x, y) \mid y \sim p_{\theta}(\cdot|x)\}$. This step

ensures that the subsequent teacher guidance is grounded directly in the student’s current exploration space, effectively mitigating the distribution shift often observed in standard KD.

Teacher Logits via Packed Parallel Prefill. Unlike on-policy methods that require synchronized teacher generation, our decoupled framework allows the teacher to act as a pure evaluator. Crucially, this enables the application of sequence packing during the annotation phase. We concatenate multiple student responses into fixed-length sequences, allowing the teacher to process them via parallel forward pass (prefill) rather than costly autoregressive decoding. For each position t in a sequence y , the teacher computes the probability distribution conditioned on the prefix $y_{<t}$. To balance storage efficiency with distillation quality, we record only the top- k logits rather than the full vocabulary distribution. This strategy preserves the teacher’s distributional uncertainty within the student’s relevant search space, providing rich, fine-grained supervision while maximizing computational resource utilization.

KD Train with a Composite Loss. In the training phase, the student model is updated by minimizing a composite loss function that jointly incorporates standard supervised learning and knowledge distillation. The total objective $\mathcal{L}_{\text{total}}$ is formulated as a weighted combination of the Cross-Entropy (CE) loss and KD loss:

$$L_{\text{total}} = (1 - \lambda) \cdot L_{\text{CE}} + \lambda \cdot L_{\text{KD}}, \quad (1)$$

where $\lambda \in [0, 1]$ is a hyperparameter regulating the trade-off between the hard-label supervision ($\mathcal{L}_{\text{CE}}$) and the teacher’s soft distributional guidance.

To focus the distillation on the most plausible tokens and filter out long-tail noise, we adopt a Top- k distillation objective. Let $\mathcal{V}_{\text{top-}k} \subset \mathcal{V}$ denote the subset of indices corresponding to the k largest logits of the teacher model. The distillation loss is reformulated as the KL divergence computed exclusively over this high-confidence support:

$$\mathcal{L}_{\text{KD}} = \sum_{c \in \mathcal{V}_{\text{top-}k}} p_{\text{T}}(c) \log \frac{p_{\text{T}}(c)}{p_{\text{S}}(c)}, \quad (2)$$

Minimizing this objective aligns the student’s predictive distribution with the teacher’s dominant predictions, encouraging the student to capture key semantic information while ignoring negligible probability mass in the tail.

3.2 Adaptive Data Selection via Δ -IFD

In the near-policy setting, response data is generated by the student model itself. While these student-generated outputs align closely with the student’s intrinsic distribution, they inevitably introduce quality noise compared to ground-truth data. To empirically filter out noise while retaining training signals that drive improvement, we introduce a heuristic sample selection based on the IFD Li et al. [2024], Zhou et al. [2024] metric.

IFD Definition. IFD quantifies the semantic alignment between an instruction Q and a response A by contrasting the unconditioned loss against the conditioned loss. Formally, let $\mathcal{L}_{\theta}(\cdot)$ denote the cross-entropy loss of the sequence. The IFD score is defined as:

$$\text{IFD}(Q, A) = \frac{\mathcal{L}_{\theta}(A \mid Q)}{\mathcal{L}_{\theta}(A)}, \quad (3)$$

Intuitively, a lower IFD score indicates higher model confidence and better alignment, whereas a score $\text{IFD} > 1$ implies that the instruction fails to facilitate—or even hinders—the prediction of A .

Student-generated Output (SGO)		
Input: Provide a list of successful community initiatives supported by the city council member's sibling. Answer with at least 3 bullet points.		
Output: * Initiative 1 \n* Initiative 2 \n* Initiative 3.		
Teacher IFD: 0.007	Student IFD: 0.018	Δ-IFD: -0.01 < 0
Input: Which city serves as the capital of Australia?		
Output: The capital of Australia is **Canberra**.(...)		
Teacher IFD: 0.46	Student IFD: 0.21	Δ-IFD: 0<0.25<1
Input: 哪位哲学家是《哲学的贫困》这本书的作者？		
Output: 《哲学的贫困》这本书的作者是**卡尔·马克思**（Karl Marx）。\n\n这本书是马克思在1967年发表的...		
Teacher IFD: 1.43	Student IFD: 0.15	Δ-IFD: 1.28 > 1

Figure 2: Illustration of Data Curation Zones defined by Δ -IFD.

Differential Difficulty (Δ -IFD). To capture the *relative learnability* of a sample, we propose the differential IFD metric, denoted as Δ_{IFD} . This metric measures the cognitive gap between the teacher’s evaluation and the student’s confidence:

$$\Delta_{\text{IFD}} = \text{IFD}_{\text{Teacher}} - \text{IFD}_{\text{Student}}, \quad (4)$$

where $\text{IFD}_{\text{Student}}$ represents the student’s intrinsic uncertainty (lower implies higher confidence), and $\text{IFD}_{\text{Teacher}}$ reflects the teacher’s acceptability of the reasoning path. Based on Δ_{IFD} and a difficulty threshold τ , we categorize student-generated samples into three distinct zones, as shown in Fig 2:

- **The Degenerate Zone ($\Delta_{\text{IFD}} < 0$):** Although student-generated data typically aligns better with the student’s own distribution (i.e., $\text{IFD}_{\text{Student}} < \text{IFD}_{\text{Teacher}}$), a negative differential ($\text{IFD}_{\text{Teacher}} < \text{IFD}_{\text{Student}}$) presents an anomaly where the teacher exhibits higher confidence than the student. As illustrated in the first row of Figure 2, this counter-intuitive phenomenon typically arises from *short, meaningless responses* (e.g., trivial patterns or empty sequences). Since the teacher has seen such frequent patterns during pre-training, it assigns them low loss, whereas the student generates them with high uncertainty due to policy degradation. We classify these as degenerate noise and exclude them.
- **The Cognitive Disconnect Zone ($\Delta_{\text{IFD}} > \tau$):** A differential exceeding the threshold τ indicates that the *teacher model exhibits extreme uncertainty* regarding the response (i.e., $\text{IFD}_{\text{Teacher}} \gg \text{IFD}_{\text{Student}}$). This highlights a severe cognitive discrepancy: while the student generates the content with high confidence, the teacher evaluates it as highly improbable or erroneous. Including such conflicting data creates a substantial mismatch that exerts a detrimental impact on the training process, forcing high-variance optimization steps and destabilizing the training trajectory.
- **The Proximal Learning Zone ($0 \leq \Delta_{\text{IFD}} \leq \tau$):** We target samples where the teacher confirms base quality ($\text{IFD}_{\text{Teacher}} \leq 1$) and the difficulty gap is moderate. These samples represent “low-hanging fruit”—challenges that are slightly above the student’s current competence but strictly within reach. Distilling logits from this zone serves as a stable learning signal, ensuring the student model updates safely within its current capacity without being overwhelmed by noise.

Consequently, our final training set is constructed by retaining only samples located within the proximal learning zone. Crucially, this heuristic filtering serves as a robust stabilizer against the policy lag inherent in asynchronous frameworks. Even if the data-generating policy drifts from the current training state, the Δ -IFD criterion effectively intercepts extreme out-of-distribution samples. Empirically (Fig 5), without Δ -IFD, noisy samples trigger high-variance updates, driving the KL divergence to 0.12. In contrast, Δ -IFD effectively intercepts out-of-distribution noise, suppressing the divergence within 0.1. This prevents noise-dominated gradients and stabilizes the optimization trajectory, ensuring updates safely remain within the proximal learning zone.

3.3 Constraining the RL Search Space

Reinforcement Learning inherently struggles with expansive search spaces, often resulting in slow convergence and instability during the cold-start exploration phase. While standard protocols typically initiate RL from a SFT checkpoint to mitigate this, we demonstrate that the NPD framework yields a significantly superior initialization.

Our analysis reveals that the distilled model functions as more than a mere weight initialization; it fundamentally *reshapes* the initial policy distribution. By training exclusively on high-quality data filtered via Δ_{IFD} , the student’s probability mass becomes highly concentrated on a manifold of logically correct and instruction-aligned trajectories prior to RL. This effective *pre-conditioning* drastically narrows the exploration space, enabling the subsequent RL stage to bypass inefficient stochastic exploration and focus immediately on fine-grained optimization, thereby unlocking superior asymptotic performance.

Table 1: Comparison of accuracy and training time between NPD and other distillation methods. The experiments are conducted by distilling the teacher model **Qwen2.5-Math-7B-Inst** ($\mathcal{M}_T$) into the student model **Qwen2.5-Math-1.5B** ($\mathcal{M}_S$). Train Time represents the NPU hours consumed per epoch. Results annotated with $\dagger$ are taken from DistiLLM-2.

Method	GSM8K	MATH500	Train Time
$\mathcal{M}_T$ (Teacher)	89.31 $\dagger$	80.00	–
$\mathcal{M}_S$ (Student)	77.33 $\dagger$	62.00	–
Supervised KD	78.62	75.00	1.17
SeqKD	82.41	72.60	2.06
GKD	80.17	73.60	6.54
DistiLLM	81.15	73.80	12.00
DistiLLM-2	81.27	74.20	12.00
NPD	82.87	76.60	1.48

4 Experiments

4.1 Experimental Setups

Knowledge Distillation Setup. We employ openPangu-Embedded-7B Chen et al. [2025] as the teacher model to distill knowledge into the openPangu-Embedded-1B student model. The training is conducted on the SFT dataset for 10 epochs. We utilize the AdamW optimizer with a weight decay of 0.1 and a gradient clipping threshold of 1.0. To maximize computational efficiency, we implement sample packing with a maximum sequence length of 32,768 tokens. The global batch size is set to 2 M tokens, and the learning rate follows a cosine decay schedule ranging from 1×10^{-5} to 1×10^{-6} . Unless otherwise stated, all experiments follow these default configurations.

Evaluation. We evaluate our models on a diverse suite of 11 benchmarks categorized into four primary domains: (1) **General Capabilities** are assessed via MMLU, CMMLU Li et al. [2023], C-Eval Huang et al. [2023], IF-Eval Zhou et al. [2023], and CLUEWSC Xu et al. [2020]. (2) **Mathematics** is evaluated using GSM8K and MATH-500 Hendrycks et al. [2021]. (3) **Reasoning** capabilities are tested on DROP Dua et al. [2019] and GPQA-Diamond Rein et al. [2024]. (4) **Code Generation** is measured using MBPP Austin et al. [2021] and HumanEval Chen et al. [2021].

4.2 Comparison with Other Distillation Methods

To ensure a rigorous and comprehensive evaluation, we benchmark NPD against two distinct distillation paradigms: the efficiency-oriented Sequence-Packed Distillation and the performance-oriented Sample-Wise Distillation.

Comparison with Sample-Wise Distillation. Due to the prohibitive computational costs associated with sample-wise baselines, we conduct controlled experiments on a 50k-instance subset of MetaQA Yu et al. [2023]. We employ Qwen2.5-Math-7B Yang et al. [2024] as the teacher and Qwen2.5-Math-1.5B as the student, training all methods for one epoch to ensure strict experimental fairness. As detailed in Table 1, our method demonstrates superiority in both performance and speed. On the GSM8K and Math500 benchmarks, NPD not only surpasses the strong baseline DistiLLM-2 Ko et al. [2025] in accuracy but also delivers a substantial efficiency gain. Specifically, under identical training conditions, NPD achieves a $8.1\times$ training speedup compared to DistiLLM-2, validating its capability to deliver on-policy-level performance with off-policy-level efficiency.

Comparison with Sequence-Packed Distillation. We further evaluate the scalability of NPD in a large-scale setting using an internal SFT dataset comprising 1.2 million high-quality samples.

Table 2: Comparison of NPD and sequence-packed distillation methods on a large-scale real-world industrial dataset. AVG denotes the average score across 11 evaluation benchmarks.

Method	AVG
SFT	56.28
Supervised KD Sanh et al. [2019]	58.79
SeqKD Kim and Rush [2016a]	59.55
NPD	64.37

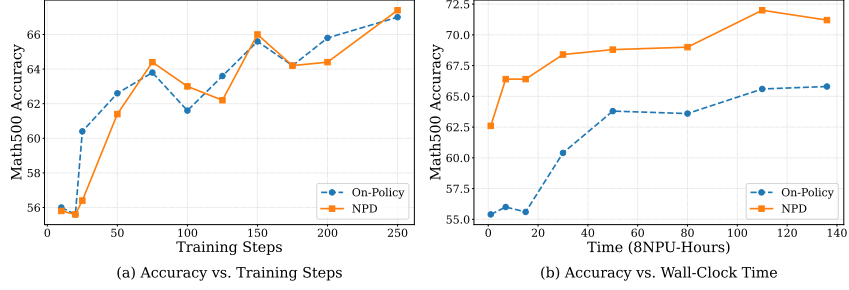


Figure 3: Comparison of NPD and On-Policy Distillation on Math500. **(a)** While both methods exhibit similar convergence trends, Near-Policy demonstrates superior data efficiency and maintains a consistent performance advantage. **(b)** Under fixed computational resources, Near-Policy achieves a substantial performance margin over On-Policy distillation.

For this experiment, we utilize openPangu-Embedded-7B Chen et al. [2025] as the teacher and openPangu-Embedded-1B as the student. As shown in Table 2, NPD demonstrates superior scalability compared to standard approaches. Crucially, the performance advantage of NPD is significantly amplified when applied to this larger-scale, real-world dataset. It yields substantial gains, outperforming the SFT baseline by 8.09% and the standard SeqKD approach by 4.83%. These results, further detailed in Table 8, confirm that NPD effectively bridges the gap between the efficiency of sequence packing and the distribution-matching benefits of interactive distillation.

4.3 Efficiency and Performance with NPD

To rigorously evaluate the advantages of NPD, we conduct a comparative study against standard on-policy distillation on the same dataset. We assess performance under two distinct constraints: (1) **sample efficiency** (fixed training steps) and (2) **computational efficiency** (fixed wall-clock time).

Performance under Fixed Steps. Benefiting from SFT-compatible architecture, NPD enables sequence packing, which allows for significantly higher effective token throughput per step compared to the sample-wise padding often required by on-policy methods. As illustrated in the left of Fig 3, under identical training steps, the accuracy trajectories on Math500 for both NPD and on-policy methods exhibit a highly consistent upward trend. This observation is critical: it demonstrates that NPD effectively retains the *optimization quality* of on-policy methods. Despite the asynchronous nature of our data generation, the student’s policy updates remain as precise and effective as those derived from synchronous, strictly on-policy gradients.

Performance under Fixed Resources. The practical advantage of NPD becomes even more pronounced when controlling for computational resources (i.e., identical wall-clock time). As shown in the right panel of Fig 3, NPD’s performance curve consistently surpasses the on-policy baseline. By decoupling the generation phase and pre-computing Student-Generated Outputs, NPD eliminates the computational bottleneck associated with real-time, step-by-step sampling. Leveraging the inherent throughput efficiency of the SFT architecture, NPD processes significantly more data within the same time window, thereby establishing a substantial performance margin over on-policy baselines in real-world training scenarios.

Table 3: Ablation study of data filtering strategies within NPD. The best result is **bolded**.

Filtering Metric	Selection Scope	AVG
NPD (Unfiltered)	–	61.63
Perplexity (PPL)	Weighted	61.69
IFD _{Teacher}	[0, 1]	62.69
IFD _{Student}	[0, 1]	62.60
Δ_{IFD} (Single-side constraint)	$(0, \infty)$	61.77
Δ_{IFD} (Single-side constraint)	$(-\infty, 1)$	61.76
Δ_{IFD} (Combined constraint)	[0, 1]	62.78

4.4 Ablation on Δ -IFD Filtering Strategy

We conduct comprehensive ablation studies to evaluate the effectiveness of our data filtering mechanism.

Comparison of Filtering Metrics. To validate the superiority of Δ_{IFD} , we benchmark it against standard filtering strategies, including Perplexity (PPL) Jelinek et al. [1977], Student-IFD, and Teacher-IFD. As detailed in Table 3, relying solely on absolute metrics yields sub-optimal results: PPL (61.69%) tends to favor shorter, simpler sequences, while raw IFD scores ($\text{IFD}_{\text{Student}}$: 62.60%, $\text{IFD}_{\text{Teacher}}$: 62.69%) fail to capture the student’s specific learning needs. In contrast, our proposed Δ_{IFD} achieves the highest baseline score of 62.78%. This empirical evidence confirms that neither absolute confidence nor absolute difficulty is sufficient; capturing the *relative knowledge gap* is critical for preventing policy lag and ensuring effective knowledge transfer in asynchronous distillation.

Constraint Analysis and Stability. Building on the efficacy of Δ_{IFD} , we further analyze its internal constraints. As shown in Table 3, we compare the performance impact of applying individual criteria ($\Delta_{\text{IFD}} < 0$ or $\Delta_{\text{IFD}} > 1$) against the combined constraint ($0 \leq \Delta_{\text{IFD}} \leq 1$). The results demonstrate that the combined strategy yields a 1.2% improvement in accuracy over the baseline. This validates that removing both “degenerate noise” ($\Delta_{\text{IFD}} < 0$) and “cognitive overload” ($\Delta_{\text{IFD}} > 1$) is essential for optimal performance.

Furthermore, as illustrated in Fig 4, our filtering scheme effectively suppresses *loss spikes* inherent to unfiltered training. By purging samples where the student is statistically overconfident or the teacher is uncertain, the $0 \leq \Delta_{\text{IFD}} \leq 1$ criterion significantly stabilizes the optimization landscape, leading to smoother convergence. Finally, sensitivity analysis (Table 5) further identifies $\tau = 0.8$ as the optimal threshold for balancing data quantity and quality.

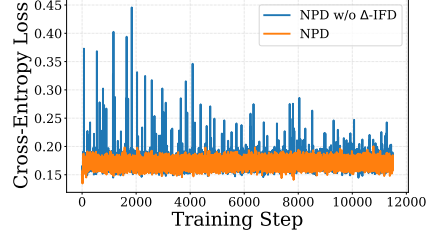


Figure 4: Comparison of training loss with and without Δ_{IFD} data filtering.

Table 4: Performance and computational cost trade-off with different update frequencies. The best performance is highlighted in bold.

Method	Update Freq.	AVG	NPU-Hours
NPD	0	62.78	335
	1	64.37	659
	2	64.17	983

Table 5: Ablation study of filtering strategies exploring threshold sensitivity, defined by the condition $0 \leq \Delta_{\text{IFD}} \leq \tau$. The best result is **bolded**.

Filtering Strategy	AVG
$\tau = 0.9$	62.86
$\tau = 0.8$	63.05
$\tau = 0.7$	62.60

4.5 The Efficacy of Sparse Updates

Table 4 examines the critical trade-off between update frequency and computational overhead. Defining the update frequency as the number of policy refreshes per 10-epoch cycle, we observe a distinct pattern of *diminishing marginal returns*. Specifically, a single strategic update (at epoch 5) yields a significant 1.59% performance boost, whereas increasing the frequency to two or more leads to saturation or even slight regression. This phenomenon suggests that excessive synchronization not only incurs higher computational costs but may also introduce unnecessary perturbations that disrupt student learning stability.

These empirical findings strongly validate our “**Sparse Updates**” hypothesis. Contrary to strict on-policy paradigms (e.g., PPO) that demand frequent, high-cost updates to minimize distributional shift, our results demonstrate that the theoretical upper bound of on-policy optimization can be effectively approximated via *sparse, periodic updates*. Consequently, we establish that frequent synchronization is not strictly necessary; instead, a strategic low-frequency schedule achieves a superior *Pareto frontier*, reconciling high training efficacy with maximum system efficiency.

4.6 Benefits for Subsequent RL

To validate the hypothesis that NPD serves as a superior initialization for Reinforcement Learning, we conduct comparative experiments using Group Relative Policy Optimization (GRPO) Guo et al. [2025]. We initiate RL training from two distinct baselines: the standard SFT model and our NPD model, using a dataset of 7,000 mathematical problems.

Table 6: **Performance comparison of RL fine-tuning outcomes.** We utilize the categorization style to group benchmarks by domain. “AVG” denotes the macro-average across all 11 datasets. Gains in parentheses indicate improvement over the respective initialization baselines.

Method	General					Math		Reasoning		Code		AVG
	MMLU	CMMLU	C-Eval	IF-Eval	CLUEWSC	GSM8K	MATH	DROP	GPQA	MBPP	HumanEval	
<i>SFT Initialization</i>												
SFT Baseline	63.21	53.10	58.51	56.38	76.95	70.89	56.20	28.73	43.43	52.53	59.15	56.28
+ GRPO (300 steps)	63.43	53.65	57.15	54.16	77.05	77.26	72.00	45.64	43.94	56.81	57.32	59.86 (+3.58)
<i>NPD Initialization (Ours)</i>												
NPD Baseline	66.44	55.43	66.73	65.43	80.94	78.24	72.60	45.69	50.51	58.37	67.68	64.37
+ GRPO (300 steps)	68.04	56.53	67.13	60.81	82.17	87.60	84.76	69.18	48.48	61.87	69.51	68.73 (+4.36)

Table 7: Instruct model (non-thinking) comparison between openPangu Embedded-1B-RL and other representative models across a diverse set of benchmarks for evaluating language and reasoning skills. **Bold** values represent the best results in each line among models at the 1B-parameter scale. If the original paper reports the results, we present the results from the original paper (marked with asterisks *); otherwise, we list our reproduced results.

Benchmark (Metric)		Qwen3	Qwen2.5	Gemma3	Llama3.2	Qwen3	MiniCPM4	openPangu-Embedded		
								SFT	KD	RL
# Total Params		1.7B	1.5B	1B	1B	0.6B	0.5B	1B	1B	1B
General	MMLU (Acc)	63.37	52.84	37.49	32.19	44.24	55.55*	63.21	66.44	68.04
	CMMLU (Acc)	61.22	53.98	31.57	10.81	42.94	65.22*	53.10	55.43	56.53
	C-Eval (Acc)	61.00*	59.30	32.49	32.08	42.60*	66.11*	58.51	66.73	67.13
	IF-Eval (Prompt Strict)	68.20*	42.50*	51.57	39.37	54.50*	50.28	56.38	65.43	60.81
	CLUEWSC (Acc)	77.36	74.59	50.20	52.36	50.31	49.90	76.95	80.94	82.87
Math	GSM8K (Acc)	77.03	73.20*	57.16	36.39	59.29	52.08*	70.89	78.24	87.60
	MATH-500 (Acc)	73.00*	46.60	39.40	18.20	55.20*	29.60*	56.20	72.60	84.76
Reasoning	DROP (F1)	61.21	48.44	30.98	45.23	34.69	30.07	28.73	45.69	69.18
	GPQA-Diamond (Pass@1)	28.60*	29.80*	19.20*	29.29	22.90*	28.28	43.43	50.51	48.48
Code	MBPP (Pass@1)	60.70	63.20*	58.75	40.08	46.69	59.14*	52.53	58.37	61.87
	HumanEval (Pass@1)	68.90	61.60*	40.24	29.88	40.85	46.34*	59.15	67.68	69.51
Average		63.69	55.10	40.82	33.26	44.93	48.42	56.28	64.37	68.73

As shown in Table 6, under identical training budgets (300 steps), the NPD initialization yields significantly higher performance gains. While the SFT-based model achieves a modest improvement of 3.58%, the NPD-based model achieves a substantial gain of 4.36%. Specifically, on mathematical benchmarks, the NPD-initialized model boosts performance by 6.52% on GSM8K and 15.00% on Math500. Furthermore, we observe notable *positive transfer* to out-of-domain tasks: despite the RL training focusing exclusively on math, the model exhibits improvements in general knowledge (MMLU +1.6%) and code generation (MBPP +3.50%, HumanEval +1.83%).

4.7 Compare with SOTA Models

We evaluate openPangu Embedded-1B on both reasoning and general language tasks by applying the combined NPD and RL framework to the openPangu-1B backbone. As detailed in Table 7, the model exhibits a clear evolutionary trajectory: starting from an SFT baseline of 56.28%, NPD optimizes the initial policy distribution to reach 64.37%, and subsequent RL refinement further unlocks reasoning potential, propelling the final score to 68.73%. This stepwise improvement validates that our framework effectively maximizes the potential of lightweight models, establishing a new performance benchmark for the 1B scale.

A salient finding is its aggregate performance, achieving an average score of 68.73%, significantly outperforming the larger Qwen3-1.7B model (63.69%). This result highlights exceptional parameter efficiency, suggesting that advanced training and alignment methodologies can be more impactful than merely scaling model size. The model’s advantage is most pronounced in mathematical and complex reasoning, where it secures leading scores on GSM8K (87.60%) and MATH-500 (84.76%). These strengths are complemented by robust general knowledge capabilities, as evidenced by top-tier performance on benchmarks such as CLUEWSC (82.87%) and MMLU (68.04%).

5 Conclusion and Discussion

We introduce NPD, a robust framework that effectively navigates the efficiency-quality trade-off by synergizing asynchronous generation with sparse student updates and Δ -IFD filtering. By integrating NPD with RL, we achieve SOTA performance with a lightweight 1B-parameter model, openPangu Embedded-1B-RL, which outperforms existing baselines of comparable scale. Reconciling high-performance AI with edge constraints, NPD offers a scalable path toward ubiquitous on-device intelligence and future edge optimizations.

6 Limitation

While Near-Policy Distillation (NPD) offers significant gains in efficiency and performance, the student model’s efficacy remains inherently constrained by the teacher’s distribution. Specifically, any biases or suboptimal logits provided by the teacher during complex reasoning steps may lead to the propagation of these systemic errors to the student.

7 Impact Statement

Near-Policy Distillation makes LLM alignment more compute-efficient by decoupling generation from training and enabling sequence packing, while retaining key benefits of on-policy distillation. This can lower the cost of training and deploying better-aligned smaller models. However, higher efficiency may also speed up scaling and deployment, so careful dataset governance and transparent filtering (e.g., Δ -IFD) are important to reduce bias and misuse. We hope this work encourages more researchers to explore scalable near-policy training pipelines and principled data selection for efficient alignment.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*, 2024a.
- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. 2024b. URL <https://arxiv.org/abs/2306.13649>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Cristian Bucila, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In *Knowledge Discovery and Data Mining*, 2006. URL <https://api.semanticscholar.org/CorpusID:11253972>.
- Hanting Chen, Yasheng Wang, Kai Han, Dong Li, Lin Li, Zhenni Bi, Jinpeng Li, Haoyu Wang, Fei Mi, Mingjian Zhu, et al. Pangu embedded: An efficient dual-system llm reasoner with metacognition. *arXiv preprint arXiv:2505.22375*, 2025.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mo Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021. URL <https://api.semanticscholar.org/CorpusID:235755472>.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *North American Chapter of the Association for Computational Linguistics*, 2019. URL <https://api.semanticscholar.org/CorpusID:67855846>.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. 2025. URL <https://arxiv.org/abs/2306.08543>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiron Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Fanchao Qi, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *ArXiv*, abs/2305.08322, 2023. URL <https://api.semanticscholar.org/CorpusID:258685666>.

- Fred Jelinek, Robert L Mercer, Lalit R Bahl, and James K Baker. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, 62(S1): S63–S63, 1977.
- Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 1317–1327, 2016a.
- Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. 2016b. URL <https://arxiv.org/abs/1606.07947>.
- Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. Distillm: Towards streamlined distillation for large language models. 2024. URL <https://arxiv.org/abs/2402.03898>.
- Jongwoo Ko, Tianyi Chen, Sungnyun Kim, Tianyu Ding, Luming Liang, Ilya Zharkov, and Se-Young Yun. Distillm-2: A contrastive approach boosts the distillation of llms. 2025. URL <https://arxiv.org/abs/2503.07067>.
- Haonan Li, Yixuan Zhang, Fajri Koto, Yifei Yang, Hai Zhao, Yeyun Gong, Nan Duan, and Timothy Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese. *arXiv preprint arXiv:2306.09212*, 2023.
- Ming Li, Yong Zhang, Zhitao Li, Jiuhai Chen, Lichang Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and Jing Xiao. From quantity to quality: Boosting llm performance with self-guided data selection for instruction tuning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7602–7635, 2024.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- Yehui Tang, Yichun Yin, Yaoyuan Wang, Hang Zhou, Yu Pan, Wei Guo, Ziyang Zhang, Miao Rang, Fangcheng Liu, Naifu Zhang, et al. Pangu ultra moe: How to train your big moe on ascend npus. *arXiv preprint arXiv:2505.04519*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Liang Xu, Hai Hu, Xuanwei Zhang, Lu Li, Chenjie Cao, Yudong Li, Yechen Xu, Kai Sun, Dian Yu, Cong Yu, et al. Clue: A chinese language understanding evaluation benchmark. *arXiv preprint arXiv:2004.05986*, 2020.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2023.
- Hang Zhou, Yehui Tang, Haochen Qin, Yujie Yang, Renren Jin, Deyi Xiong, Kai Han, and Yunhe Wang. Star-agents: Automatic data optimization with llm agents for instruction tuning. *Advances in Neural Information Processing Systems*, 37:4575–4597, 2024.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

A Algorithmic Details of Near-Policy Distillation

The complete algorithmic flow of the Near-Policy Distillation (NPD) framework is detailed in Algorithm 1. Specifically, the pseudocode illustrates the cyclical process of asynchronous trajectory generation, rigorous sample filtering via Δ -IFD, and the subsequent high-throughput optimization using sequence packing. This decoupled design ensures that both the student and teacher models operate at maximum hardware utilization while mitigating the policy lag inherent in asynchronous updates.

Algorithm 1 Iterative Near-Policy Distillation (NPD) with Δ -IFD

```

1: Input: Teacher model  $p_T$ , Student model  $p_S^\theta$ , Prompt dataset  $\mathcal{D}_x$ , Pack length  $L$ 
2: Hyperparameters: Learning rate  $\eta$ , Update interval  $K$ , Loss weights  $\lambda$ 
3: Initialize: Student parameters  $\theta_S$ , Replay buffer  $\mathcal{D}_{\text{train}} \leftarrow \emptyset$ 
4: for epoch  $e = 1$  to  $E$  do
5:   if  $(e - 1) \pmod{K} = 0$  then
6:     # Phase 1: Asynchronous Data Refresh
7:     Generate student dataset:  $\mathcal{D}_S \leftarrow \{(x, y) \mid x \in \mathcal{D}_x, y \sim p_S^\theta(\cdot \mid x)\}$ 
8:      $\Delta$ -IFD Filtering:  $\mathcal{D}_{\text{select}} \leftarrow \{(x, y) \in \mathcal{D}_S \mid \Delta_{\text{IFD}}(x, y) \text{ is valid}\}$ 
9:     Sequence Packing:  $\mathcal{S}_{\text{pack}} \leftarrow \text{Pack}(\mathcal{D}_{\text{select}}, \text{Length} = L)$ 
10:    # Phase 2: Teacher Annotation
11:    Compute Teacher Logits (no grad):  $\mathcal{O}_T \leftarrow p_T(\mathcal{S}_{\text{pack}})$ 
12:    Update Buffer:  $\mathcal{D}_{\text{train}} \leftarrow \{(S, O_T) \mid S \in \mathcal{S}_{\text{pack}}, O_T \in \mathcal{O}_T\}$ 
13:  end if
14:  # Phase 3: Student Optimization
15:  for minibatch  $(S, O_T) \in \mathcal{D}_{\text{train}}$  do
16:    Compute Student Logits:  $O_S \leftarrow p_S^\theta(S)$ 
17:    Compute Probs:  $p_T \leftarrow \text{Softmax}(O_T), p_S \leftarrow \text{Softmax}(O_S)$ 
18:    Compute KD Loss:  $\mathcal{L}_{\text{KD}} \leftarrow \sum_{c \in \mathcal{V}_{\text{top-}k}} p_T(c) \log \frac{p_T(c)}{p_S(c)}$ 
19:     $\mathcal{L}_{\text{total}} \leftarrow (1 - \lambda)\mathcal{L}_{\text{CE}}(S) + \lambda\mathcal{L}_{\text{KD}}$ 
20:    Update:  $\theta_S \leftarrow \theta_S - \eta \nabla_{\theta} \mathcal{L}_{\text{total}}$ 
21:  end for
22: end for
23: Return Optimized  $\theta_S$ 

```

B Comprehensive Ablation Studies and Hyperparameter Analysis

B.1 KD Method Ablation

As shown in Table 8, we benchmark several sequence-packed distillation strategies under the same teacher–student setting (**openPangu-Embedded-7B** $\rightarrow$ **openPangu-Embedded-1B**) on a 1.2M-sample real-world industrial dataset, with all models trained for 10 epochs. The results indicate that vanilla SFT provides a solid baseline but yields limited improvements on harder capabilities such as math, reasoning, and code. Both supervised KD and SeqKD bring consistent gains over SFT, suggesting that richer teacher supervision and sequence-level packing can enhance the student’s generalization across diverse tasks. Notably, our **NPD** method achieves the best overall performance (AVG=64.37) and delivers broad improvements across most benchmarks, demonstrating its effectiveness and robustness in large-scale industrial distillation.

Table 8: Comprehensive comparison between NPD and sequence-packed distillation methods. The experiments utilize **openPangu-Embedded-7B** as the teacher and **openPangu-Embedded-1B** as the student. Evaluations are conducted on a large-scale real-world industrial dataset comprising 1.2 million samples, with all models trained for 10 epochs.

Method	General					Math		Reasoning		Code		AVG
	MMLU	CMMLU	C-Eval	IF-Eval	CLUEWSC	GSM8K	MATH-500	DROP	GPQA	MBPP	HumanEval	
SFT	63.21	53.10	58.51	56.38	76.95	70.89	56.20	28.73	43.43	52.53	59.15	56.28
Supervised KD	62.68	53.97	58.83	59.70	77.56	72.10	65.20	30.24	46.46	57.20	62.80	58.79
SeqKD	61.74	54.94	62.83	59.89	79.92	74.30	61.20	36.59	47.98	55.25	60.37	59.55
NPD (Ours)	66.44	55.43	66.73	65.43	80.94	78.24	72.60	45.69	50.51	58.37	67.68	64.37

B.2 NPD Component Ablation

We perform a step-by-step ablation to isolate the contributions of each component. As shown in Table 9, transitioning from standard KD to Packed Distillation based on student responses provides the initial structural baseline jump. However, our two novel components (Δ -IFD and Sparse Updates) provide crucial, cumulative gains necessary to reach state-of-the-art performance.

While the shift to student-generated packing establishes the foundation, the **Sparse Update** mechanism contributes the most among our novel additions (+1.59), as it physically resets the policy lag and prevents asynchronous training collapse. The Δ -IFD filter provides an additional significant boost (+1.15) by strictly pruning harmful variance.

Table 9: Ablation study of different modules in NPD. This table illustrates the cumulative performance gains by progressively integrating key components: Packed Distillation, the Δ -IFD filtering mechanism, and the Sparse Update strategy. The results demonstrate that while student-response-based packing establishes a robust baseline, our proposed Δ -IFD and Sparse Update are crucial for reaching state-of-the-art performance.

Method / Component	AVG	Δ Improvement
Standard KD	58.79	–
Packed Distillation (Student Rollouts)	61.63	+ 2.84
+ Δ -IFD Filter	62.78	+ 1.15
+ Sparse Update (Full NPD)	64.37	+ 1.59

B.3 Sensitivity to Update Frequency

Our framework demonstrates high robustness and is largely insensitive to the specific update frequency. As shown in Table 10, both the full NPD and the variant without the Δ -IFD filter (NPD w/o IFD) achieve optimal performance with just one to two sparse updates (64.37 and 63.43, respectively).

Theoretically, a policy update should be dynamically triggered whenever the KL divergence approaches our predefined ϵ boundary (e.g., 0.10). However, our extensive empirical evaluations reveal that uniformly allocating 1 or 2 updates across a standard 10-epoch training cycle naturally intercepts the critical points just before severe policy drift occurs. Therefore, in practical deployments, it is not necessary to incur the computational overhead of dynamic monitoring; statically allocating one to two updates consistently yields optimal returns.

Table 10: Sensitivity analysis of the update frequency (τ). Both configurations peak at 1 or 2 updates, demonstrating that minimal interventions are sufficient to prevent policy lag.

Update Frequency (τ)	NPD (Full)	NPD w/o IFD
0	62.78	61.63
1	64.37	63.26
2	64.17	63.43
3	–	63.12

B.4 NPD Train Hyperparameter Ablation

We conduct a systematic ablation study on knowledge distillation to further enhance model performance. Our analysis focuses on two key dimensions: (i) the weighting of the distillation loss term, and (ii) the effect of the top- k value during decoding.

KD Loss Weight. To optimize the balance between the standard cross-entropy loss and the distillation loss, we conduct experiments with different values of the KD loss weight (λ_{KD}), ranging from 0.5 to 1.0. To accelerate the experimental cycle, we conduct distillation experiments based on a single stage (Direct Fast Response) of SFT. This approach introduces a new Stage 2 to perform distillation guided by labels, with the distillation learning rate kept consistent with that used in SFT.

This systematic comparison shows setting $\lambda_{KD} = 0.9$ yields the best overall performance, as shown in Table 11.

Table 11: Effect of KD loss weight on the average benchmark performance. The evaluation metric is the average zero-shot accuracy across eight benchmarks. All models use greedy decoding with a decode length of 8K.

λ_{KD}	0.5	0.6	0.7	0.8	0.9	1.0
Average	53.94	54.07	53.88	54.43	55.29	54.44

Top- k Value Effect. To explore the impact of the number of top tokens used in the distillation process, we experiment with four values for the top- k parameter: 5, 10, 15 and 20. The experimental setup is consistent with that described in the KD Loss Weight section. We observe that training time remains almost identical across different k values. In terms of performance, increasing k from 5 to 10 yields an improvement of 0.51%, with top- $k=10$ achieving the highest average accuracy. However, further enlarging k to 15 or 20 leads to a degradation in performance, as shown in Table 12. Hence, we adopt $\lambda_{KD} = 0.9$ and top- $k=10$ by default unless otherwise specified.

Table 12: Effect of top- k value on the average benchmark performance. All models use greedy decoding with a decode length of 8K, and the KD loss weight is fixed at $\lambda_{KD} = 0.9$.

Top- k	5	10	15	20
Average Performance	54.78	55.29	54.24	54.40

C Theoretical and Empirical Analysis

C.1 Analysis of Policy Latency

To establish Near-Policy as a rigorous conceptual bridge, we conduct a quantitative analysis of policy latency. We track the Kullback-Leibler divergence, $D_{KL}(\pi_{\text{learner}} \parallel \pi_{\text{generator}})$, between the active learner and the rollout generator throughout the training process. As shown in Fig. 5, off-policy training exhibits severe distribution drift, with the KL divergence rapidly diverging to 0.30. In contrast, our proposed NPD method (green line) strictly maintains the KL divergence below 0.10. Furthermore, our sparse update mechanism (blue line) acts as a forced policy synchronization. When a weight sync occurs (e.g., around step 6900), the KL divergence sharply drops from 0.09 to 0.05, effectively resetting the policy lag.

Based on our empirical results, we formally define the **Near-Policy** regime: At any training step t , the distance between the current student policy ($\pi_{\text{current}}^{(t)}$) and the data-generating policy (π_{gen}) is strictly bounded by a small threshold ϵ :

$$D_{KL}(\pi_{\text{current}}^{(t)} \parallel \pi_{\text{gen}}^{(t)}) \leq \epsilon$$

Unlike strictly **on-policy** training ($\epsilon = 0$), which is computationally prohibitive, or **off-policy** training ($\epsilon \rightarrow \infty$), which leads to divergence, NPD dynamically enforces this boundary. Our results show that NPD empirically maintains a strict boundary of $\epsilon \approx 0.10$.

C.2 Justification for Δ -IFD

Although Δ -IFD may appear heuristic at first glance, it fundamentally serves as a measure of *local policy alignment*. Since responses are actively sampled from the student’s current policy, the student-side IFD is expected to satisfy a natural lower-bound property. When $\Delta\text{-IFD} < 0$ (i.e., Teacher IFD < Student IFD), a theoretical *statistical anomaly* arises: the student becomes paradoxically less confident in its own sampled response than the teacher. This indicates a mismatch between the sampled trajectory and the student’s local policy geometry, suggesting that such samples may be unreliable for stable optimization.

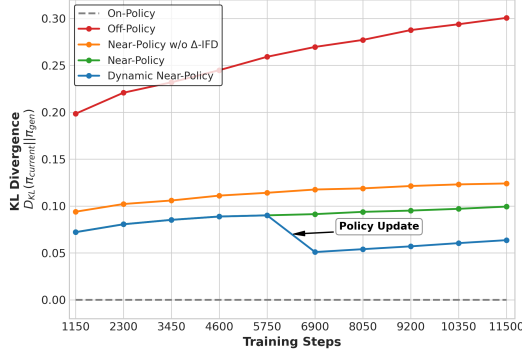


Figure 5: KL divergence over training steps. Unlike standard off-policy methods that suffer from severe drift, NPD maintains strict divergence bounds (< 0.10), with sparse updates ensuring forced policy synchronization.

As shown in Table 13, approximately 96% of rollouts are safely retained, while only a small fraction of anomalous samples are filtered out. Importantly, this filtered 4% corresponds to a high-variance and potentially poisonous long tail. Enforcing updates on these anomalous samples can lead to explosively compounding gradient variance and, in turn, destabilize training. Therefore, our design emphasizes *precision over recall*: we prefer conservative filtering to reliably prevent catastrophic drift.

Table 13: Filtering ratio of different rollout zones on the 12M proprietary dataset.

Zone	Early (%)	Late (%)
Degeneration	2.60	3.77
Proximal Learning	96.20	96.19
Cognitive Disconnect	1.20	0.04

C.3 Search Space Reduction

To provide quantitative evidence for the search-space reduction effect, we analyze the autoregressive generation space of LLMs, which closely resembles the standard reinforcement learning setting where unconstrained optimization often leads to high-variance updates. In contrast, NPD actively prunes this space and restricts the optimizer to a high-reward sub-manifold.

KL Divergence. Our updated KL plot shows that unconstrained trajectories diverge rapidly. In comparison, Δ -IFD imposes a strict bound on the optimization trajectory, empirically demonstrating that learning is confined to a narrow trust region.

Causal Mechanism. Removing Δ -IFD significantly weakens the RL gains, reducing the improvement from +4.36 in full NPD to only +2.48 (see Table 14). This directly supports our claim that spatial pruning is a key mechanism for obtaining a higher-quality initialization and enabling more effective downstream RL optimization.

Table 14: Effect of Δ -IFD on initialization quality and downstream GRPO improvement.

Method	AVG
NPD w/o Δ -IFD Init	63.43
+ GRPO (300 steps)	65.91 (+2.48)
NPD Init (Ours)	64.37
+ GRPO (300 steps)	68.73 (+4.36)

D Extended Evaluations and Scalability

D.1 Scalability to Larger Model Capacities

To evaluate the scalability of our approach, we apply the NPD framework to larger-capacity models. Specifically, we distill a 72B teacher model into a 7B student model (openPangu-7B) using our internal 1.2M dataset. As shown in Table 15, NPD yields significant improvements over the SFT baseline across multiple challenging benchmarks. This indicates that NPD scales effectively and maintains its performance advantages under larger parameter regimes.

Table 15: Scalability experiments on a 7B student model. We compare the standard SFT baseline with our NPD framework when distilling from a 72B teacher.

Method	AVG	GPQA	AIME24	AIME25	LiveCodeBench
SFT	38.32	63.64	25.21	35.00	29.41
NPD (Ours)	41.73	69.36	32.08	33.12	32.35

D.2 Thinking Models

Beyond the fast-thinking base models used in our main study, we further evaluate NPD on a reasoning model setting using **openPangu-1B-think** as the student and **openPangu-7B-think** as the teacher, with the context window extended to 32K tokens.

As shown in Table 16, NPD delivers strong gains across diverse benchmarks and improves the overall average score from 66.43 to 73.52. Notably, even at a 32K context length, NPD still yields a large gain of **+7.09 AVG**, further validating its effectiveness for long-form reasoning models.

Table 16: Performance on a 1B student model with thinking mode with 32K context length.

Method	MMLU	CMMLU	C-Eval	GSM8K	MATH-500	MBPP	HumanEval	GPQA	DROP	CLUEWSC	IFeval	AVG
SFT	70.61	57.61	63.08	78.32	73.00	60.70	60.37	46.46	75.92	79.92	64.70	66.43
NPD (Ours)	79.31	62.81	63.57	88.10	77.00	78.99	82.93	49.49	79.94	84.43	62.11	73.52

D.3 Evaluation on Code Generation Tasks

To further validate the efficacy of NPD, we conduct experiments in the coding domain using Qwen2.5-Coder-7B-Inst (Teacher) and Qwen2.5-Coder-1.5B (Student) on the *WizardCoder* dataset. As summarized in Table 17, NPD consistently outperforms existing baselines, demonstrating its robustness and generalization capabilities across diverse domains.

Table 17: Performance comparison on coding benchmarks. NPD establishes a new state-of-the-art among baseline methods.

Method	HumanEval	MBPP	AVG
$\mathcal{M}_T$ (Teacher)	75.61	74.60	75.10
$\mathcal{M}_S$ (Student)	30.73	60.84	45.78
GKD	40.85	61.90	51.38
DistiLLM	39.63	62.17	50.90
DistiLLM-2	42.24	62.70	52.47
NPD (Ours)	43.58	63.01	53.29

D.4 Multi-Path Reasoning Expansion

To further push performance boundaries, we introduce an optional multi-path reasoning strategy. When computational budgets permit, we extend the student’s inference process from generating a single greedy path to sampling K independent trajectories for each instruction. This expansion significantly broadens sample diversity, allowing the model to explore a wider region of its potential

solution space. Crucially, this approach synergizes effectively with our Δ -IFD filtering mechanism to establish a robust “*generate-then-filter*” paradigm: by rigorously applying the Δ_{IFD} criterion, we ensure that the distillation process benefits from a richer variety of reasoning patterns while strictly excluding low-quality noise.

Table 18 presents a stepwise ablation to decouple these contributions. Starting from the baseline (61.63%), introducing Multi-Path Reasoning ($K = 3$: 1 greedy, 2 sampled) boosts accuracy to 62.82%. However, this generation overhead significantly increases training time. Subsequently, applying our Δ_{IFD} filter to these diverse trajectories further elevates performance to 63.29%. This validates that while increasing sample diversity is beneficial, rigorously filtering out low-quality noise is essential for stabilizing optimization.

Table 18: **Ablation study on the effectiveness of Multi-Path Reasoning and Δ_{IFD} Filtering.** We progressively incorporate the multi-path generation strategy and the dynamic filtering mechanism. **Train Time** indicates the computational overhead in NPU hours.

Multi-Path	Δ_{IFD}	Train Time (h)	AVG (%)
		335	61.63
✓		1000	62.82
✓	✓	998	63.29

E End-to-End Training Efficiency and Cost Analysis

To provide full transparency regarding computational costs, we profile the end-to-end time breakdown for training NPD over 10 epochs on the 1.2M dataset (using an 8-NPU node):

Table 19: End-to-end time breakdown for training NPD.

Phase	Time (8-NPU Node, h)	Percentage
Student Rollout	300	82.01%
Teacher Annotation	18	4.92%
Δ -IFD Computation	5	1.37%
Filtering	0.04	0.01%
Packing	0.75	0.21%
Training	42	11.48%

As shown in Table 19, our Δ -IFD filtering introduces negligible overhead (1.38%), while Student Rollout dominates the cost (82.01%). Traditional strict on-policy methods necessitate synchronous inference and optimization (i.e., a “one-rollout-per-update” paradigm), which severely underutilizes expensive training NPUs by leaving them in prolonged idle states. In contrast, NPD not only fully decouples the student rollout process asynchronously—enabling horizontal scaling with additional resources to accelerate inference—but also implements a “single-rollout-multiple-updates” strategy for data utilization. This architectural design substantially conserves overall computational resources and maximizes the system’s training throughput.